

Практическое задание №1

Docker & Build & Docker Compose

1. Докеризовать каждый прикладной сервис (собственный Dockerfile).
2. Поднять весь стек через docker compose.
3. Настроить межсервисную сеть и переменные окружения.
4. Настроить nginx-gateway как единую точку входа.
5. Обеспечить работоспособность пользовательских сценариев: регистрация, логин, витрина, корзина, заказ, кабинет, обработка заказа воркером.

Введение

Цель работы: Развертывание микросервисной архитектуры приложения Microshop с использованием технологий контейнеризации.

В ходе выполнения работы были выполнены следующие шаги:

1. **Контейнеризация:** Перевод прикладных сервисов на стеках Go, Java, Python и React в изолированные Docker-контейнеры.
2. **Оркестрация:** Объединение разрозненных сервисов в единую экосистему с помощью docker compose.
3. **Сетевое взаимодействие:** Настройка межсервисных коммуникаций внутри виртуальной сети Docker, обеспечение безопасности и изоляции.
4. **Единая точка входа:** Настройка Nginx в качестве API-Gateway для проксирования запросов к фронтенду, API сервисам и медиа-хранилищу.
5. **Интеграция инфраструктурных решений:** Развертывание и связывание компонентов хранения данных (PostgreSQL), кэширования (Redis), очередей сообщений (RabbitMQ), хранилища пользователей (etcd) и объектного хранилища (MinIO).

Ход работы

Для каждого микросервиса был написан Dockerfile.

Dockerfile для микросервиса auth-service-go:

```
1 >> FROM golang:1.23-alpine AS builder
2     WORKDIR /app
3     COPY go.mod go.sum ./
4     RUN go mod download
5     COPY . .
6     RUN go build -o /app/auth-service ./cmd/auth-service/main.go
7
8     FROM alpine:latest
9     WORKDIR /app
10
11     COPY --from=builder /app/auth-service .
12     COPY --from=builder /app/config.yml .
13
14     EXPOSE 8081
15     CMD ["/app/auth-service"]
```

Dockerfile для микросервиса backend-java:

```
1 >> FROM maven:3.9-eclipse-temurin-21-alpine AS builder
2     WORKDIR /app
3     COPY pom.xml .
4     RUN mvn dependency:go-offline
5     COPY src ./src
6     RUN mvn clean package -DskipTests
7
8     FROM eclipse-temurin:21-jre-alpine
9     WORKDIR /app
10    COPY --from=builder /app/target/*.jar app.jar
11    EXPOSE 8080
12    CMD ["java", "-jar", "app.jar"]
```

Dockerfile для микросервиса frontend-react:

```
1 >> FROM node:18-alpine AS builder
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm install
5 COPY . .
6 RUN npm run build
7
8 FROM nginx:alpine
9 COPY --from=builder /app/dist /usr/share/nginx/html
10 COPY nginx.conf /etc/nginx/conf.d/default.conf
11 EXPOSE 80
12 CMD ["nginx", "-g", "daemon off;"]
```

Dockerfile для микросервиса nginx-gateway:

```
1 >> FROM nginx:alpine
2 COPY nginx.conf /etc/nginx/conf.d/default.conf
3 EXPOSE 80
```

Dockerfile для микросервиса order-worker-python:

```
1 >> FROM python:3.11-slim
2 ENV PYTHONUNBUFFERED=1
3 WORKDIR /app
4 COPY requirements.txt .
5 RUN pip install --no-cache-dir -r requirements.txt
6 COPY . .
7
8 CMD ["python", "worker.py"]
```

Файл `compose.yml` является центральным узлом управления всей экосистемой. Для обеспечения отказоустойчивости и корректного порядка запуска сервисов были использованы механизмы `depends_on` с проверкой состояния `healthcheck`.

postgres: Основная реляционная БД. Использование `healthcheck` гарантирует, что `backend-service` и `order-worker` не начнут попытки соединения до полной готовности СУБД:

```
postgres:
  image: postgres:15-alpine
  environment:
    POSTGRES_DB: microshop
    POSTGRES_USER: user
    POSTGRES_PASSWORD: password
  volumes:
    - pgdata:/var/lib/postgresql/data
    - ./infra/postgres-init:/docker-entrypoint-initdb.d:ro
  healthcheck:
    test: [ "CMD-SHELL", "pg_isready -U user -d microshop" ]
    interval: 5s
```

redis: Обеспечивает быстрое хранение сессий для `auth-service` и кэширование данных каталога для `backend-service`:

```
redis:
  image: redis:7-alpine
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
```

rabbitmq: Брокер сообщений, создающий канал связи между Java-бэкендом и Python-воркером.

```
redis:
  image: redis:7-alpine
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
```

etcd: Хранилище конфигураций и данных пользователей для `auth-service` использующее REST API для взаимодействия.

```
etcd:
  image: quay.io/coreos/etcd:v3.5.0
  command: /usr/local/bin/etcd --advertise-client-urls http://etcd:2379 --listen-client-urls http://0.0.0.0:2379
  environment:
    - ALLOW_NONE_AUTHENTICATION=yes
```

minio: S3-совместимое хранилище. Для автоматизации мы добавили сервис minio-init, который при запуске автоматически создает бакеты и загружает тестовые изображения товаров, делая стенд «саморазворачиваемым»

```
minio:
  image: minio/minio
  environment:
    - MINIO_ROOT_USER=minioadmin
    - MINIO_ROOT_PASSWORD=minioadmin
  command: server /data --console-address ":9001"
  volumes:
    - minio_data:/data

minio-init:
  image: minio/mc
  depends_on:
    - minio
  volumes:
    - ./infra/minio-init/init.sh:/init.sh
    - ./infra/generated-images/products:/seed/products
  environment:
    - MINIO_ROOT_USER=minioadmin
    - MINIO_ROOT_PASSWORD=minioadmin
  entrypoint: /bin/sh /init.sh
```

Каждый сервис был упакован в контейнер с учетом принципа «**один процесс — один контейнер**». Это позволило достичь:

- **Изоляции зависимостей:** Сервис на Java (Backend) не конфликтует с сервисом на Go (Auth), так как каждый работает в собственной файловой системе с необходимыми ему библиотеками.
- **Воспроизводимости:** Благодаря использованию Dockerfile, окружение приложения теперь идентично на любом компьютере, где установлен Docker, что исключает проблему работы на одной машине
- **Оптимизации размеров:** Использование образов alpine и slim позволило значительно снизить вес итоговых образов, что критично для скорости развертывания в облачных средах.

В ходе выполнения работы поднимается контейнер.

☐		devops:practice1-docker:compose-main	-	-	-	206.67%	2 minutes ago		:	
☐		gateway-1	3986064f358a	devops-practice1-docker	3000:80	0%	2 minutes ago		:	
☐		order-worker-1	7ec9259e353f	devops-practice1-docker		0%	2 minutes ago		:	
☐		backend-service-1	e64f3a9ed4f9	devops-practice1-docker		0.17%	2 minutes ago		:	
☐		auth-service-1	4b7ac274afe	devops-practice1-docker		0%	3 minutes ago		:	
☐		minio-init-1	6dfa4f2a008a	minio/mc		0%	3 minutes ago		:	
☐		postgres-1	5a33abfdd74c	postgres:15-alpine		0%	3 minutes ago		:	
☐		rabbitmq-1	82e933b60d63	rabbitmq:3-management	15672:15672	205.78%	3 minutes ago		:	
☐		redis-1	107d838bade6	redis:7-alpine	Show all ports (2)	0.38%	3 minutes ago		:	
☐		frontend-1	319eb1fbf20a	devops-practice1-docker		0%	3 minutes ago		:	
☐		minio-1	e57cd7cb5829	minio/minio		0.03%	3 minutes ago		:	
☐		etcd-1	32efd819fc48	coreos/etcd:v3.5.0		0.31%	3 minutes ago		:	

С помощью панели админа можно добавить новый товар:

NovaShop

ВитринаЛичный кабинетАдмин-панель

ВыйтиКорзина: 0

Вы вошли как 234

Laptop Pro

1299.99 USD

Кол-во: 1

В КОРЗИНУ

Headphones X

199.99 USD

Кол-во: 1

В КОРЗИНУ

Mouse

49.99 USD

Кол-во: 1

В КОРЗИНУ

семечки

50 USD

Кол-во: 1

В

Также работает сервис доставки, через некоторое время заказ доставляется:

Вы вошли как 234

Личный кабинет

#10

•

семечки

•

577 шт.

•

Заказ доставлен в магазин по адресу Университетская площадь, д.1

Заключение.

В ходе лабораторной работы была успешно спроектирована и реализована микросервисная архитектура платформы «Microshop». Выполнение задачи позволило на практике применить современные DevOps-практики и методы контейнеризации для создания надежной, масштабируемой системы.

Данная работа позволила глубоко понять механизмы межсервисного взаимодействия в изолированных сетях Docker, принципы работы reverse-проxy и важность правильной настройки healthcheck для предотвращения ошибок старта системы. Навыки, полученные при отладке взаимодействия между Java-бэкендом, Python-воркером и инфраструктурными компонентами, являются фундаментальными для работы в области системного и платформенного администрирования.